

CLASS 12

**Kawchar Husain**

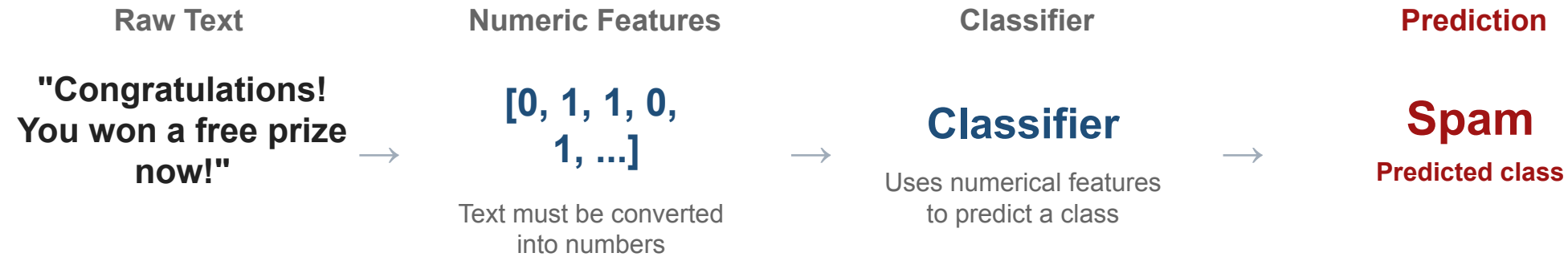
ML Engineer • Kaggle Master
ICPC Asia-West Finalist

Naive Bayes & NLP Fundamentals

From probability to text classification

How Can a Model Classify Text?

Text contains useful signals, but ML models need numerical features.



Words like "won", "free", and "prize" can help classify the message.

Basic Probability

Probability measures how often an event occurs

$P(A) = \text{outcomes in the event} / \text{total outcomes}$

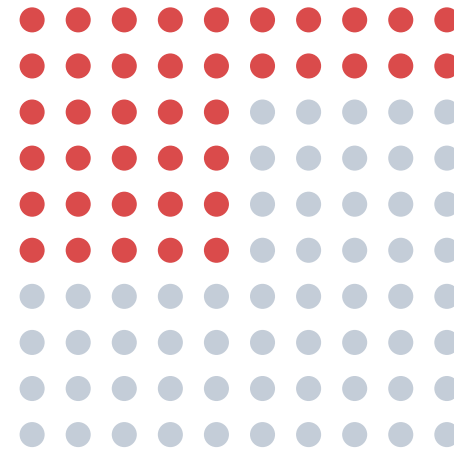
Example: 100 people

40 sick

60 not sick

$$P(\text{Sick}) = 40 / 100 = 0.40$$

The denominator is the full group: all 100 people.



red dots = sick

gray dots = not sick

Conditional Probability

New information changes the group we are reasoning about

Given: the person is a **smoker**

Now we only consider the 50 smokers.

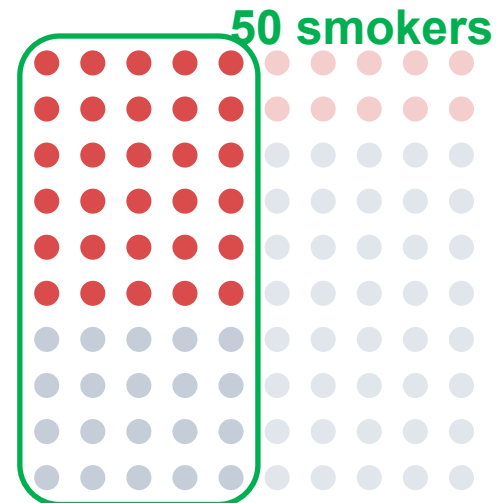
$$P(\text{Sick} \mid \text{Smoker}) = 30 / 50 = 0.60$$

General formula

$$P(A \mid B) = P(A \cap B) / P(B)$$

$$\begin{aligned} P(\text{Sick} \mid \text{Smoker}) &= P(\text{Sick} \cap \text{Smoker}) / P(\text{Smoker}) \\ &= (30 / 100) / (50 / 100) = 30 / 50 = 0.60 \end{aligned}$$

| means “given” $\cap$ means “and / both”



30 sick smokers

same 100 people as previous slide

Bayes' Theorem Reverses the Condition

Use what we know to calculate the probability we actually want

Start from conditional probability

$$P(A | B) = P(A \cap B) / P(B)$$

$$P(B | A) = P(A \cap B) / P(A)$$

Rearrange:

$$P(A \cap B) = P(A | B) \times P(B)$$

$$P(A \cap B) = P(B | A) \times P(A)$$

Both expressions equal $P(A \cap B)$, so:

$$P(A | B) \times P(B) = P(B | A) \times P(A)$$

Divide by $P(B)$:

$$P(A | B) = P(B | A) \times P(A) / P(B)$$

Bayes' Theorem in Classification

Which class is most probable after observing the features?

In classification: A = the class we want to predict, B = the features we observe

$$P(A | B) = P(B | A) \times P(A) / P(B)$$

$$\mathbf{P(\text{Class} | \text{features}) = P(\text{features} | \text{Class}) \times P(\text{Class}) / P(\text{features})}$$

P(Class | features) — After seeing these features, how likely is this class?

P(features | Class) — How common are these features inside this class?

P(Class) — How common is this class in the training data?

P(features) — How common are these features overall?

For the same observed features, P(features) is the same denominator for every class.
So, to decide which class is larger, compare only:

$$\mathbf{P(\text{features} | \text{Class}) \times P(\text{Class})}$$

But how do we calculate P(features | Class) when there are many features?

Naive Bayes Assumption

Treat features as conditionally independent once the class is known

The hard part is calculating the joint probability of many features.

Without the assumption

$$P(x_1, x_2, x_3 | y)$$

Estimate combinations together

Naive
assumption

Given the class y

$$P(x_1 | y) \times P(x_2 | y) \times P(x_3 | y)$$

Treat feature probabilities as independent

$$P(x_1, x_2, \dots, x_n | y) \approx P(x_1 | y) \times P(x_2 | y) \times \dots \times P(x_n | y)$$

“Naive” refers to this conditional-independence assumption.

Naive Bayes Classification Formula

Combine Bayes' theorem with the independence assumption

From Bayes in classification

$$P(y | X) \propto P(X | y) \times P(y)$$

Using the naive assumption

$$P(X | y) \approx P(x_1 | y) \times P(x_2 | y) \times \dots \times P(x_n | y)$$

Final Naive Bayes score

$$\text{score}(y) = P(y) \times P(x_1 | y) \times P(x_2 | y) \times \dots \times P(x_n | y)$$

Prediction: calculate this score for every class and choose the largest one.

$$\hat{y} = \underset{y}{\operatorname{argmax}} \ P(y) \times \prod_i P(x_i | y)$$

Let's Build a Naive Bayes Classifier

Use the training table to predict a new day

Training data

Day	Weather	Temp	Play
1	Sunny	Mild	Yes
2	Sunny	Hot	No
3	Rainy	Mild	Yes
4	Sunny	Mild	Yes
5	Cloudy	Mild	Yes
6	Rainy	Hot	No
7	Sunny	Cool	No
8	Cloudy	Hot	Yes
9	Rainy	Mild	Yes
10	Sunny	Mild	No

New observation

Weather = Sunny

Temperature = Mild

Predict: Play = Yes or Play = No

We will first learn probabilities from the training table, then use only the needed ones for this new day.

Training Step 1: Learn Class Probabilities

Count how common each class is in the training data

Training data

Day	Weather	Temp	Play
1	Sunny	Mild	Yes
2	Sunny	Hot	No
3	Rainy	Mild	Yes
4	Sunny	Mild	Yes
5	Cloudy	Mild	Yes
6	Rainy	Hot	No
7	Sunny	Cool	No
8	Cloudy	Hot	Yes
9	Rainy	Mild	Yes
10	Sunny	Mild	No

Count the Play labels

Play = **Yes**: 6 rows

Play = **No**: 4 rows

Class probabilities learned during training:

$$P(\text{Yes}) = 6 / 10 = 0.60$$

$$P(\text{No}) = 4 / 10 = 0.40$$

These class probabilities are stored and reused during prediction.

Training Step 2: Learn Feature Probabilities

Calculate every feature value inside every class

Training data

Day	Weather	Temp	Play
1	Sunny	Mild	Yes
2	Sunny	Hot	No
3	Rainy	Mild	Yes
4	Sunny	Mild	Yes
5	Cloudy	Mild	Yes
6	Rainy	Hot	No
7	Sunny	Cool	No
8	Cloudy	Hot	Yes
9	Rainy	Mild	Yes
10	Sunny	Mild	No

Weather probabilities

Weather	P(value Yes)	P(value No)
Sunny	2 / 6	3 / 4
Rainy	2 / 6	1 / 4
Cloudy	2 / 6	0 / 4

Example: among 6 Yes rows, Sunny appears 2 times, so $P(\text{Sunny} | \text{Yes}) = 2/6$.

Temperature probabilities

Temp	P(value Yes)	P(value No)
Mild	5 / 6	1 / 4
Hot	1 / 6	2 / 4
Cool	0 / 6	1 / 4

Denominators are class counts: Yes = 6 rows, No = 4 rows.

Training calculates probabilities for every feature value, not only the values needed for the next prediction.

Some feature-class combinations have zero counts: $P(\text{Cloudy} | \text{No}) = 0$ and $P(\text{Cool} | \text{Yes}) = 0$.

Inference: Use the Needed Probabilities

For a new sample, retrieve the learned values and compare scores

New day

Weather = Sunny
Temperature = Mild

Naive Bayes score formula

$$\text{Score}(y) = P(y) \times P(\text{Sunny} \mid y) \times P(\text{Mild} \mid y)$$

Then substitute the values learned during training.

For y = Yes

$$\begin{aligned}\text{Score}(\text{Yes}) &= P(\text{Yes}) \times P(\text{Sunny} \mid \text{Yes}) \times P(\text{Mild} \mid \text{Yes}) \\ &= 6/10 \times 2/6 \times 5/6 \\ &\approx 0.167\end{aligned}$$

For y = No

$$\begin{aligned}\text{Score}(\text{No}) &= P(\text{No}) \times P(\text{Sunny} \mid \text{No}) \times P(\text{Mild} \mid \text{No}) \\ &= 4/10 \times 3/4 \times 1/4 \\ &= 0.075\end{aligned}$$

$0.167 > 0.075 \rightarrow \text{Predict Play} = \text{Yes}$

The same denominator $P(\text{Sunny}, \text{Mild})$ applies to both classes, so comparing scores is enough.

Types of Naive Bayes

Choose the variant that matches the feature type

Naive Bayes variants differ by the kind of feature values they model.

Variant	Feature type	Typical example
Gaussian NB	Continuous numeric values	height, salary, sensor reading
Categorical NB	Discrete categories	weather, color, category
Multinomial NB	Counts or non-negative frequencies	word counts, TF-IDF features
Bernoulli NB	Binary present / absent values	word present = 1, absent = 0

Weather example: Categorical NB logic. Text classification: MultinomialNB with CountVectorizer / TF-IDF.

Choose the variant based on what the feature values represent.

NLP Fundamentals

Turning human language into representations a model can learn from

Natural Language Processing (NLP) helps computers work with human language.

Raw Text



Preprocessing



Numerical Representation

BoW • TF-IDF • Word Embeddings

Text Preprocessing

Clean and standardize text before extracting features

Raw text — FREE prize!!! Click NOW.

Lowercase — free prize!!! click now.

Remove punctuation — free prize click now

Tokenize — [free, prize, click, now]

Optional stopwords removal — [free, prize, click]

Not every NLP task needs every preprocessing step; choose steps based on the model and problem.

Stemming vs. Lemmatization

Two ways to reduce word variations

Stemming

Rule-based chopping. Fast, but may produce non-words.

word	stem
studies	studi
wolves	wolv
running	run

Lemmatization

Uses vocabulary and grammar. Slower, but returns valid base forms.

word	lemma
studies	study
wolves	wolf
better	good

Stemming is mechanical; lemmatization is more language-aware.

Vectorization Converts Text into Numbers

Different methods represent text in different ways

Example document: “free prize free now”

Bag of Words

Vocabulary: [free, prize, now, meeting]

[2, 1, 1, 0]

Document vector of raw counts
“free” appears twice

TF-IDF

Same vocabulary

[TF-IDF(free), TF-IDF(prize),
TF-IDF(now), 0]

Weighted document vector
Weights depend on the full corpus

Word2Vec

A vector for each word

free → [0.12, -0.41, ...]
prize → [0.27, -0.18, ...]
now → [-0.05, 0.31, ...]

Dense learned embeddings
One vector per word

BoW and TF-IDF create document feature vectors. Word2Vec learns embeddings for individual words.

Bag of Words Counts Words

Build a vocabulary, then count word occurrences in each document

Three short documents

D1: free free prize now

D2: free offer now

D3: team meeting now

Step 1: collect the vocabulary

Step 2: count each word in each document

Document-term matrix

Doc	free	prize	offer	team	meeting	now
D1	2	1	0	0	0	1
D2	1	0	1	0	0	1
D3	0	0	0	1	1	1

Columns = vocabulary words | Rows = documents

Bag of Words ignores word order; it keeps only the count of each vocabulary word.

Multinomial Naive Bayes Learns from Word Counts

Counts are used to estimate $P(\text{word} \mid \text{Class})$

Training word counts

Class	free	prize	offer	meeting
Spam	8	5	6	0
Not Spam	0	0	0	9

Total word count in Spam = $8 + 5 + 6 + 0 = 19$

Total word count in Not Spam = $0 + 0 + 0 + 9 = 9$

Convert counts to probabilities

Before smoothing:

$$P(\text{free} \mid \text{Spam}) = 8 / 19$$

$$P(\text{prize} \mid \text{Spam}) = 5 / 19$$

$$P(\text{offer} \mid \text{Spam}) = 6 / 19$$

$$P(\text{prize} \mid \text{Not Spam}) = 0 / 9 = 0$$

For the message “free prize offer”:

$$\text{Score}(\text{Spam}) = P(\text{Spam}) \times P(\text{free} \mid \text{Spam}) \times P(\text{prize} \mid \text{Spam}) \times P(\text{offer} \mid \text{Spam})$$

Score(Not Spam) contains **$P(\text{prize} \mid \text{Not Spam}) = 0$**

Therefore the whole Not Spam score becomes **0**.

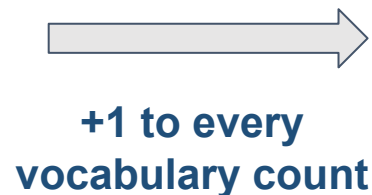
This zero-probability problem is why we use smoothing.

Laplace Smoothing Fixes Zero Probabilities

Add 1 to every vocabulary count so no word probability becomes zero

Before smoothing

Class	free	prize	offer	meeting
Spam	8	5	6	0
Not Spam	0	0	0	9



After smoothing

Class	free	prize	offer	meeting
Spam	9	6	7	1
Not Spam	1	1	1	10

General formula

$$P(\text{word} \mid \text{Class}) = (\text{count}(\text{word}, \text{Class}) + 1) / (\text{total word count in Class} + V)$$

V = number of words in the vocabulary

Example: word “prize” inside the Not Spam class

Here: $\text{count}(\text{prize}, \text{Not Spam}) = 0$ | $\text{total word count in Not Spam} = 9$ | $V = 4$

$$P(\text{prize} \mid \text{Not Spam}) = (0 + 1) / (9 + 4) = 1 / 13$$

Smoothing prevents one zero count from making the whole Naive Bayes score zero.

TF-IDF Gives More Weight to Informative Words

Frequent in one document, rare across the collection

Same three documents as Bag of Words

D1: free free prize now

D2: free offer now

D3: team meeting now

Document frequency → IDF intuition

prize	1 of 3 documents	higher IDF
free	2 of 3 documents	medium IDF
now	3 of 3 documents	lowest IDF

$$\text{TF-IDF} = \text{Term Frequency} \times \text{Inverse Document Frequency}$$

A word that appears everywhere is less useful for distinguishing documents.

Word2Vec Learns Word Embeddings

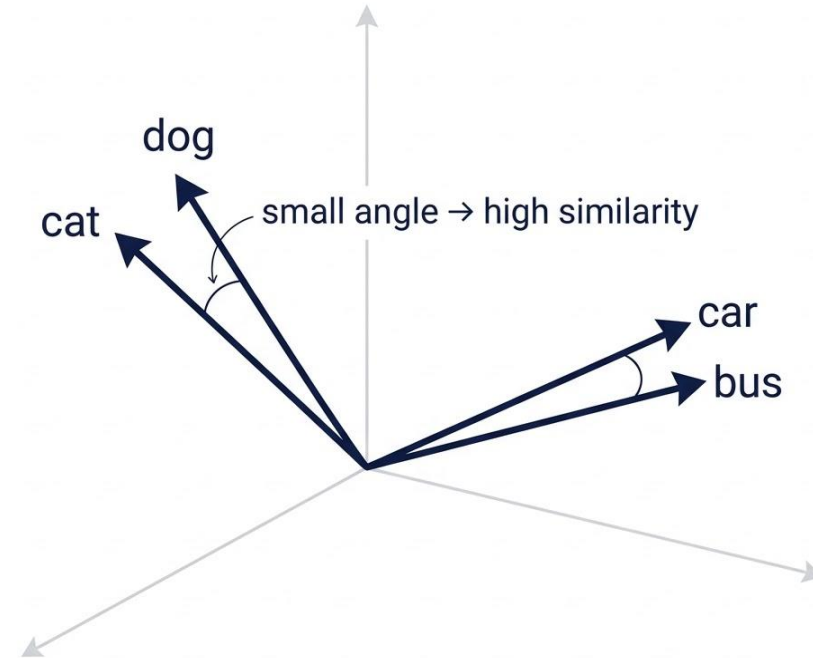
Words used in similar contexts learn similar vector representations

word → dense vector

king → [0.21, -0.63, 0.14, ...]

Each word gets a learned numeric vector.

The vector is learned from surrounding word context.



Words used in similar contexts tend to have similar vector directions.

Similarity is commonly measured using cosine similarity.

Let's Implement It with Python

CountVectorizer and MultinomialNB inside one leakage-safe pipeline

```
from sklearn.model_selection import
train_test_split
from sklearn.feature_extraction.text import
CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

X_train, X_test, y_train, y_test =
train_test_split (
    X, y, test_size=0.2, stratify=y,
    random_state=42
)

model = make_pipeline (CountVectorizer (),
MultinomialNB ())
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

Vectorizer

CountVectorizer learns the vocabulary from `X_train` and turns messages into word-count rows.

Model

MultinomialNB scores each class from those counts. Laplace smoothing is built in.

One pipeline

The vectorizer and model are fitted only on training data, then evaluated once on `X_test`.